

Course Overview

Course Overview

Hi, everyone. My name is Dan Wahlin, and I'd like to welcome you to the C# Events, Delegates, and Lambdas course. I'm a software developer, architect, and trainer, specializing in development, container, and cloud technologies. Over the years, I've had the opportunity to work with C# and .NET extensively and look forward to helping you learn additional ways to enhance your C# skills. In this course, you'll learn about core C# features, including events and event args, delegates, lambdas, and more. Some of the major topics that we'll cover include the role of delegates in events in C# apps, how to create and use custom delegates, what lambdas are and how they can be used in different scenarios, and how you can leverage built-in delegate types, such as Action of T and Func T, TResult that .NET provides out of the box. By the end course, you'll understand the role that delegates, events, and lambdas play and how you can apply them to your own .NET apps. Before beginning the course, you should be familiar with core C# concepts such as classes and methods, understand how to use a coding editor or IDE, such as Visual Studio, or VS Code, and have basic experience with the command line. I hope you'll join me on this journey to learn about events and delegates with the C# Events, Delegates, and Lambdas course, at Pluralsight.

The Role of Events, Delegates, and Event Handlers

Course Overview

Welcome to the C# Events, Delegates, and Lambdas course. My name is Dan Wahlin, and I'm really excited to walk you through these really core concepts of C# that have been around for a long time, including all the way back to the early versions of C#. So whether you're just starting out with C# and

you want to learn this or you've been working with C# for quite a while and you want to figure this out as well, either way this course will cater to you. Now, before we dive into this module, I want to go over some of the version information about the language, some prereqs, and where you can find the code, and then we'll start diving into the actual topics. Now, as far as the version of C# used for the course, you'll see that information here, but it's important to note that everything I'm going to cover goes all the way back to the beginning of C#. C# 1 had events and delegates and then lambdas came shortly after that, so this is going to be applicable virtually anywhere you can run C# code. So what are we going to cover? Well, first off, we're going to introduce the role of events, delegates, and event handlers, clarify visually what they are, why you should care about them, and how they interact with each other, and then we'll move on to actually creating those and using them in code. From there, we're going to talk about how you can handle events, some different techniques that can be used, and then we're going to wrap up with lambdas, one of my favorite topics and something you'll see in lots and lots of C# code, whether you're maintaining it or writing it yourself, and we're going to talk about some generics that are built into C# such as `Action` and `Func`, I'll explain what those are and how you would use those. So by the time you get done with this course, you'll have a very solid foundation in events, delegates, and lambdas and understand how all of those fit together and how you can actually put them to use. Now, when it comes to the prereqs for the course, as long as you have some basic programming and experience with an editor, as well as C# core concept knowledge, and this would be things like classes, properties, and methods, then you'll be fine going through this course. Now, I will show a little bit of command line along the way. You don't need to be a command-line expert, but we will see some of that as well as we move throughout the course. We're also going to be using different types of editors such as Visual Studio and VS Code. Now, the code for this course can be found here. If you go to this repo, you'll be able to grab the latest and greatest, and I would recommend you check it out fairly frequently because I'll do updates over time that will be pushed here. Now, this is one course I've done on C# and .NET, but I have a lot of other courses out there if you're interested. If you're interested, for instance, in Docker and containers or Kubernetes, if you work with front-end technologies, TypeScript, Angular, things like that, several different courses are out there, but in this course, we're obviously going to focus on just the C# angle of events, delegates, and lambdas. So let's dive into an overview of this module.

Introduction

In this module, we need to introduce some core concepts, so we're going to visually describe the role of events, the role of delegates, and the role of event handlers, and then we're going to see a little code in some demos to show how these are actually used in practice in different types of application for .NET. Now, what's really important here is just to understand the concepts. You're going to see some code, of course, but in the next module, we'll really dive into what these are and how you can actually use them from a C# standpoint. Let's do first things first and start talking about what these are. Now the analogy that I'm going to show you in just a moment, it's a very visual analogy, is of two people communicating. Every time you actually talk to a person, we could tie events, delegates, and event handlers into that communication process. Now, obviously from a real-world standpoint, we don't ever refer to it as that or even think of it that way at all, but there's lots and lots of scenarios when communication is used where these technologies are going to apply. So let's start things off by discussing the role of events and give you this real life picture of how communication plays in to these technologies.

The Role of Events

Events are really the heart and soul of any application out there. We could even talk about JavaScript running in the browser and events play a big role there. You might be building desktop apps and the user clicks on a button that fires an event, or maybe you have an object that as something occurs with the object, an order is placed, for example, you want to notify other parts of the system that something just happened. So events are used all over the place and this is the case not only with C#, but many other languages out there. Now, the analogy I mentioned earlier of communication is shown here. Let's say that we have the girl on the left and she's talking to maybe her dad on the right here and they're using the good old tin can with a string, and as they talk, we could tie in all the different parts of events, delegates, and event handlers into this. Now, in this case, since we're focusing on events to start, the little girl here is the event raiser. We're going to assume she's talking first into this, that sound is transmitted over the string, and then the person at the other end can actually listen to that and hear it. Now, we're going to tie delegates and event handlers into this as well, but once you understand this visually, it's much easier to understand how these three things tie together. So what is an event then? Well, it's a notification, it's a message that goes out. There's lots of ways you can think of it, but let's

take your phone, for example. You might actually have something that came in, such as an SMS message, an email, and you get notified. Well, that's a type of event that occurred that the operating system on that phone or maybe it's on your desktop or laptop or something like that, that it was able to process. So they play a really central role in C# apps, and it does depend on the type of app you're going to see because obviously, if you're just writing back-end apps, then you might not involve events with say button clicks, maybe that's all happening in a browser or some other mechanism. But let's assume you're writing a desktop app for C#. Well, in that case, events are not only used for object communication, but they could also be used for notifications about a drop-down item was selected, a button was clicked, those types of things, so they provide a way to trigger notifications from end users or even from objects or processes that are running in our C# application. Now, when it comes to the role of events, let's say that you have a button and the user clicks it and that generates an event. So in this case, events signaled the occurrence of an action, but you're going to see throughout the course that they can also be notifications that an object or process performs some type of action itself, a notification if you will. So whether you call it an action or a notification or just an event, in this example throughout this course, we're really talking about the same thing. Now, what's nice about events is that objects that are raising the events don't have to explicitly know about the other object that's actually listening and will handle the event. Now, what's nice about that is it means that as the button is clicked, the button could care less who's listening, or as you have an order object and it places an order out there, we could care less about who's listening. We just want the opportunity for people to listen to us if you will, but it's up to them you're going to see to actually do the listening. As events are fired, they can pass some data, and we often refer to this as event args or event data. So that's a quick look at what events are. Now, what are delegates and how do they fit into this process? We'll look at that next.

The Role of Delegates

Delegates work hand-in-hand with events. In fact, without delegates, events would be pretty useless. So let's talk about what is a delegate and how do we look at this from a visual standpoint? Let's go back to our image of the communication that's going on, and we talked about the girl was going to talk, but obviously, we have the string that is between these two tin cans that is the magic, and if you've ever tried this before, it actually works at least to some extent because the audio can be transmitted

along that string. And so we need that string because otherwise, you would just have a tin can for the girl, a tin can for the guy, and you know, it wouldn't really be very impressive, especially if you're trying to do it from a little further distance. So the string is what we would call the delegate. It's responsible for allowing the girl to be able to talk to her dad on the right, and that allows us to communicate in .NET apps as well you're going to see. So what is a delegate? Well, in a nutshell, a delegate is the glue or I like to think of it as the pipeline between an event and an event handler, just like we have the string that connects the two cans in our picture. From a C# standpoint, a delegate is a specialized class. It's based on a multicast delegate base class, and it's often referred to or called a function pointer. If you think about it, the person listening on the right is really just a function that wants to get this data, so the delegate is going to point the data that the girl is saying to the function, which would be the guy on the right and then they can do something with that data and act upon it. Let's drill down more into this pipeline concept. So we have an event on the left, we have this pipe, and then we have an event handler on the right. Now, we refer to the pipe as the delegate. It's the communication glue between the event and the event handler, and that's why I said earlier without the delegate, it would be pretty hard for this all to work. So if we have an event, it's going to pass event args, that's the data, the event handler then gets that data and then we can act upon it. Now, if we look at that from a button standpoint, when the button is clicked, the button is going to send some event args, and then the method on the other end of this pipe handles that data and can act upon it. A button doesn't really have that much useful data for event args, but something like a mouse would. If we had a mouse click event, then the event args might have the XY coordinates of where the user clicked that mouse, and then we could get to those in the method that's on the right, our event handler, and we could act upon those and do something with it. So from a visual standpoint, delegates are like pipelines. We're going to look at it more from a code standpoint coming up a little bit later in the course, but for now, this will start to set the foundation and help you visually see how events and delegates tie together. Now, the next piece of the puzzle is obviously the event handler, so let's talk about that next.

The Role of Event Handlers

The last part we need to cover is the role of event handlers. Obviously, we have the girl on the left, she talks, the string or the delegate handles that data, and then routes it or points it to the person listening on the right. Now that is the event handler, so we have now the event, we have the string or delegate,

and we have the listener, the event handler. So what is an event handler? Well, really all it is is a way to receive and process data from a delegate, so you'll see in the image on the left, we have that pipeline, the event args can flow in that pipeline, and then the event handler, which is really just a method, can then handle processing that type of data. So an event handler may receive different types of parameters, I mentioned this a little bit earlier. Normally, you'll have the sender of the event so it could be the button, the mouse click, or it could be an object that's triggering the event, and then you'll have the actual data that the event handler wants to get to. If a customer order was placed and that was the event, customer order inserted, let's say, then we probably want to get to information about the customer, the order ID, those types of things. That's what would go in the event args. So event args are responsible for encapsulating event data, and you could have a lot of data in event args. It's not just one or two things, in some cases, it could be many things actually, which typically is going to be defined using properties. So if we go back to what we looked at earlier, when a button is clicked, we know that event args are passed along through the delegate to the event handler or the method. Now that might look like the following. If it was a button click, we might have a `Submit_Click` and notice in the signature for this method, we have two parameters. We have an object sender, it's object because it could be any type of sender, it's very open-ended, and then we have the `EventArgs`. Now, I've already mentioned that with buttons, there's not a lot of information that's in `EventArgs` that's of use, but with other types of events, there definitely is useful information such as the mouse click that I talked about that might have XY coordinates and some other data. Or if this was an order inserted event, order updated event, those type of things, then the sender might be the order object, the event args might be information about that order that was inserted or updated or deleted or whatever it may be. Now, what's nice about this in C# is it's very predictable. No matter what you work with, it's going to look somewhat similar to this. The only exception might be some front-end work you might do with C#, such as Blazor, for example, running in a browser. Well, yeah, browser is going to handle events a little bit differently than say a desktop app or a console app or things like that that are pure C#, but in general, this type of format that you see here is very, very consistent. So now that we've looked at what an event is, what a pipeline or delegate is, and what an event handler is, let's put all this together and look at some demos of this in action.

Events, Delegates, and Event Handlers in Action

Let's take a look at events, delegates, and event handlers in action and start to examine some of the code that we're going to be covering in upcoming modules. So I have a Windows Forms application loaded up, and this is a very simple app just to get us started with everything we've covered throughout this module, events, delegates, and event handlers. So if we click on our combo box here, if I right-click and go to Properties, we can come down to our events, click the little lightning icon, notice that selected index changed is the event and it's pointing to an event handler. Now, likewise, if we go to the button, you'll see that the click event is pointing to an event handler, and of course, we can just double-click these to get to that event handler code, and there we have the one for the ComboBox and the one for the SubmitButton_Click. Notice that the parameters on both of these are identical. You get object sender, that would either be the combo box or the button as appropriate, and you get the EventArgs. Now, in this case, the EventArgs is just empty, there's nothing useful there, but you'll see later on in the course that can certainly change. Now, you might wonder where's the delegate? If we expand our Form1.cs, you'll see a Form1Designer.cs, and as we drag and drop, this is where it updates the coordinates of the buttons, the combo boxes, and other components that you might add, but if we drill down into this and come on down, you're going to see a SubmitButton section and a ComboBox section, and then notice for the SubmitButton, we have a SubmitButton_Click. Alright, there's the event, there's the event handler we just looked at, and there's the delegate, that's the pipeline. Now you'll notice that we're actually attaching using this special syntax += the event to the delegate, and on the left side of it, you're adding the event to that pipe and then that data can flow through the pipe and get to the event handler over here on the right, and we'll talk more about this syntax as we move along. Now, let's come on down to the combo box. Well, there's the CountriesComboBox. SelectedIndexChanged again, there's the event, we're going to attach it to the pipeline, EventHandler, and then of course, if we scroll on over, we can see the event handler and that event handler, of course, can be found back in here for both the button and the combo box. So that's everything we've now covered throughout this module. So far, we have events, we have delegates, and we have event handlers, but that's just in a Windows Forms application. Let's look at another type of application now. In this example, we have a Blazor application, and this is going to cause WebAssembly code to run directly in the browser, but we're still going to write C# to do it. If I open up our pages over here on the right in the Solution Explorer, you'll notice I have a Counter.razor, and this is pretty much the standard project output that you get if you create the Hello World type of project,

and you'll notice that there's an onclick on a button, now we're in HTML, of course, and this is going to call an event handler called IncrementCount. Now, the first thing you're going to notice is that we're actually wiring up the two using this @onclick, and then we give it the event handler, and then the event handler itself doesn't have the sender and the event args. Now, we certainly could add that, and I'll show you that in a moment, but let's go ahead and run this first, so we'll go ahead and start it up, and then we're going to go to the route of counter, and you can see that as we click on it, very fancy, it updates that for us. Now, if we wanted to go in though and actually add the event to the signature here for our parameters, we can do that. In fact, I'm just going to take this little bit of code right in the quotes and we'll replace this, and what this is going to do is go ahead and grab our event object. We have our arrow syntax in JavaScript terms, and then we're going to pass it to IncrementCount. Now, notice this and e. Okay. So this we'd have to add, so let's say object sender and let's add the event. Now, this would be a MouseEventArgs e. Now from here, we could go ahead and write this out if we'd like, so we can do a Console.WriteLine and let's just write out sender, and we'll do the same thing. Alright, let's go ahead and run this again, and this time, I'm going to go to the developer tool, so let's first go to our route, we'll go to counter, and let's go to Inspect, and we'll come down to our console and let's click on it, and you'll notice that we get some really basic information written out here, but notice that this was not the button, in this case, it's actually the Counter class. Now, we could get to the button using standard HTML DOM manipulation, but in this case, I wanted to point out that it's a little bit different when we're working in the browser, even though this is C#. The type of events, delegates, and event handlers that you work with does depend on the context of where you're running your C# code. In this case, we happen to be running with Blazor in the browser, and therefore, we're now subject to more of the browser's rules of how it does things, even though this is C# syntax. So that's another example that uses events, but is quite different. Now, let me show you one final one. The last project we're going to take a look at is C# with .NET Maui. Now, .NET Maui allows you to build and run apps on different operating systems, so it's very flexible that way, and I have just the out-of-the-box application, really no modifications here, so off to the left, you're going to notice I do have some C# files and some XAML files. Now, I'm going to go ahead and run it first, and you'll see I have a command here, and we're going to go ahead and fire up the app. So you can see that the app's loaded, it's very simple Hello World app, but as I click it, you'll notice it tracks my clicks, click one, two, three, four times, very, very simple. Keep in mind, this is full C#, so this is not a browser or anything like I looked at previously

with you. Instead, we can open up the MainPage.xaml that you see here, and you'll notice on the button that I clicked, we have a clicked event, and we have an event handler. Now, based on what I showed earlier with Windows Forms, you might say, oh, there must be a designer file or something and there's not, in this case. What's happening here is something called delegate inference. We'll talk more about this as we move along, but the click event for a button has a very specific signature, and I'll show you that in a moment, so when it builds, it could figure out the pipeline that's needed to route that click event and the event args off to this OnCounterClicked. So let's go look at the OnCounterClicked, and we'll see that right here, and you'll notice the standard suspects, object sender, EventArgs e, very, very similar to what we saw with the Windows Forms project, but the delegate is kind of magically generated for us. This, of course, increments a count, and then we update some text that is displayed in the app, and you saw that on the button itself. So what's really important to pull out of these three demos is that with standard C#, you're always going to have an event, a delegate, and an event handler, except in cases where you might be running C# in the browser, such as WebAssembly. So it does depend on the context as I mentioned a little bit earlier, but the good news here is that events are used everywhere. It doesn't matter if you're in the browser, a desktop app, we can even have events in a console app. As an object fires and does some action, we might want to be notified about that, and we'll be looking at that. So hopefully that gives you a feel for how advanced delegates and event handlers are used. Now, what we're going to do is wrap up this module, and then in the upcoming modules, we'll start to dive into some code.

Summary

Throughout this module, you've seen that certain types of C# apps rely heavily on events, delegates, and event handlers. There are certainly C# apps out there that may not need these particular features, that's important to keep in mind, but there are many, whether it's desktop apps, console apps, or others, that use events all over the place. Now, examples where you might write some C#, but not really use events much, might be an ASP.NET MVC app or a Razor Pages app because the web is going to use get requests or post requests to actually submit data from a browser back to the server, and that's an entirely different process. But even APIs might raise an event that another part of the API might act upon and then maybe update a database or update a Redis cache or whatever it may be, so knowing about this is really, really important because they can be used in many different scenarios. So

as we've gone through this module, we've learned that events provide notifications and send data using EventArgs, delegates act as the glue or pipeline between events and event handlers, and we know that event handlers receive and process EventArgs data. So now that we've seen this from a high level, it's time to start diving into the actual code to see what does this all look like when we put it together. We saw a little bit of code earlier with an event handler, but delegates are going to involve code, raising events involve code, and we're going to talk through how all these different parts work together and how you can actually learn to use them.

Creating Delegates, Events, and EventArgs

Introduction

In the previous module, you saw the role of events, delegates, and EventArgs, and we took a look at them from a high level. Well, now we're going to dive right in starting with, how do you create a delegate? What does that look like? And when it comes to defining events, what are the different options available to you as a developer when it comes to working with events and delegates? So I'm going to show you a few options that are available there. From there, we're going to talk about how we can raise events, and you're going to see that there's also a few options available here. I'm going to show you a older technique because it's still out there in the wild a lot, but I'm also going to show you a more modern newer technique that's a little less code and what you should be using nowadays. We talked about that the delegate pipeline sends data along it, and normally when it comes to events, you're going to have two parameters that are sent, you'll have the sender, and then you have the EventArgs or event data if you will. The example I gave in the previous module is a mouse-click event. Normally, you can get to data such as the XY coordinates of where the user clicked. Well, that's the EventArgs for the mouse event. What happens when you have a custom event in your own applications? How would you write that EventArgs and get that data passed along the delegate pipeline? We're going to look at that as well. So let's kick things off by talking about how can we create a delegate?

Creating a Delegate

We know that delegates are a type of pipeline, or officially, they're function pointers, but let's look at this visually again based on what we covered earlier. So we have this communication between the girl and the man, and in this case, the girl is the event raiser, we know, we know that we have a delegate which is the string or the pipe, if you will, and then we have the event handler, and ultimately, the event raiser wants to send some EventArgs up to the event handler so they can get that data. Now to kick all of this off, we need that pipeline, we need that delegate so that we can actually send the data. It would kind of be like if we had some water and we want to get it from point A to point B. Well, if you don't have a pipeline or a canal or whatever you want to call it, obviously, the water is not going to get where you want it to go, so it's very important that we get this delegate defined right up front.

Fortunately, defining a delegate is very straightforward. We can use the delegate keyword. So here's an example of one, we have a delegate and then notice everything after the delegate keyword, the int, WorkPerformedHandler, and then the parameters? That looks just like a method, right? A delegate is just a function pointer, a method pointer, it points to some type of method that, in this example, is going to get the int hours and some WorkType, in this case, WorkType you're going to see is an enum, and it's going to pass those two pieces of data along the delegate pipeline. And so if we break this down visually, we have the int and the work type that the event raiser ultimately wants to send, they're going to send that down the pipeline, and then we'll have some event handler method that can get to that data. So to kick all of this off though, you can see that we're using the delegate keyword, and then we have the signature for what the method or the event handler is actually going to look like. Again, in this case, it passes two parameters. This isn't the standard, by the way, you'll notice this isn't the sender and the EventArgs, but we'll get there. So let's jump to a simple console project and let's get this plugged in. So the first thing I'm going to do is add a delegate. Now, a delegate is technically a class, so I'm going to put this outside of my Program class that you see here, and we're going to do exactly what you saw earlier. So we're going to say delegate, it's going to return an int, we're going to call it WorkPerformedHandler, and you'll notice a standard with delegates that they'll typically have the word handler included. And then, because I've already played with this a little bit, you'll see int hours and WorkType. Now, what is WorkType? Well, it's just an enum, so I'm going to throw that in the bottom here, and it's going to look like that, so we're going to have a WorkType.GoToMeetings, WorkType.Golf, my favorite one to do, WorkType.GenerateReports. Now, we've defined a delegate at

this point, and that's really all we'd have to do, but you might be going, okay great, Dan. It looks like a method. I don't really get, why would I use this? So before we dive into events, let's see how delegates can actually be used on their own. So I'm going to come on in and create a new static int, and we're going to call this WorkPerformed1, and I'll explain why here. You'll see that through the magic of Copilot and things like that, it's actually going to try to help me out and this is exactly what I want. So we want a WorkPerformed1, we have an int hours and a WorkType, and then notice we're just going to write that out to the console and we're going to return some value, that's the int, of course, on the return type. So how would we actually use this delegate though and pause it when it's invoked to call this WorkPerformed1? Well, we can do the following. We could say var, and I'm just going to call it del1, and then notice the new WorkPerformedHandler, and it's going to give it WorkPerformed1. Alright, we're going to plug that in. Now that creates the delegate, but now we need to actually invoke it, so I'm going to do that by capturing the return value. Let's call this finalHours, and then we're going to do exactly what it says here. We're going to say del1, and then I'm going to pass an integer value, in this case, 10 and a WorkType.GenerateReports or I could do my favorite, which is Golf, if only I could find a job where I get to golf as part of the job, but anyway. Now what this is going to do is literally invoke the delegate, but notice we're calling it just like a function. Now, where is this data going to be sent? Well, of course, when we created the delegate here, we said pass that pipeline or dump the data from that pipeline into this ending point called WorkPerformed1. It takes the same parameters. Notice those match up, that's super, super important, and then once that gets invoked as we're doing here by simply calling the delegate as a function, that will actually pass this data or point it, if you will, to this WorkPerformed1, and now we get the data, so let's go ahead and run this. Alright, so it just invoked it and notice WorkPerform1 called and then 10, and so we actually get the value that we would expect, that's what we wanted. Now, what's interesting about this is you can actually create multiple instances of this delegate, so let me show you another little trick we can do here with delegates, and let's say that we had another method called WorkPerformed2, and I'm just going to copy this and maybe we write out the work type, in this case. Okay. So let's call this 2, that needs to be 2, and then let's take workType.ToString, and let's write that out. Now what's going to happen here is right up top before we actually invoke delegate one, I'm going to do a little magic, and you're going to see this with events. I'm going to do the following. I'm going to say del1 += del2, so we're doing a concatenation if you will. All we're going to do though is invoke del1, and let's see what happens here,

so let's run it again, and notice WorkPerformed1 called 10 and WorkPerformed2 called, and we could have a 3 and a 4 and a 5. So what's really cool about this is you can, in essence, chain these if you will, or we can concatenate these. Now, is this something I do a lot? No, it's actually not, but this syntax that you see right here once we get to events is going to be really, really important, and that's why I wanted to show you this += syntax. So at this point, we have our delegate defined, we instantiated it, we said where it points the data to, we even created another instance of it, tied those two together, and then we invoked the delegate which then kicks off the process of sending this data down two pipelines. Now, normally again, it may just be one, and with the events, it normally is just one, and we'll get to that as we move along throughout the module, but this is a start, it gets us thinking about what a delegate is and how we can use it, So now let's take a look at how we can use this with events.

Defining an Event

Now that we have a delegate defined, how do we go about defining an event that can actually use that delegate pipeline? That's what we're going to look at in this section. So we have a delegate in place, but we need a way for the event raiser to actually send some data up to the event handler. So how do we do that? How do we tie these two together? Well, it's actually surprisingly simple. Just like with a delegate, you use the delegate keyword, with an event, you're going to use the event keyword. So notice in this case, we're going to have a public event, the type, though, is the WorkPerformedHandler. Now that's the delegate, and then the event name is WorkPerformed, and that's all we'd have to do to define an event that's going to use the WorkPerformedHandler to actually send whatever data that handler, that delegate has defined in its signature, so let's see how we can add this in the code. So I've commented out the code we looked at in the previous section because we're going to do something different here. The first thing I'm going to do is right-click, and we're going to add a new class, and I'm going to call this class Worker. Now, I'm going to go ahead and make it public. We'll assume we might use this in other areas. We have a default namespace already defined for our project, so I'm going to go ahead and take all this out. And now what I'm going to do is define our event. Now, recall that we have our delegate from earlier called WorkPerformedHandler, so I'm actually going to copy that in my clipboard, and we're going to come back here and we're going to do what you just saw. We're going to say a public event, we're going to give it the type

WorkPerformedHandler, and then I'm going to call it WorkPerformed. Now, you're going to notice when I mouse over this, it says non-nullable event WorkPerformed, so we can fix that by adding the question mark, that operator, and now we're all set and we could work with this. What's interesting is from here, we could come on in now and let's come on down right below the comment. We could create a new worker, and when we say worker., notice what happens here, WorkPerformed. There's our event, and through the magic of some Copilot functionality, it's actually trying to give me a hint on what I could do. We're going to get to that a little later. But I just want to show you that once you add that event as a member of the class, you can then actually call that event and we can hook it up to a callback function. Now, we just did that earlier with the delegate, but we're going to do it in a slightly different way when it comes to events, but we are going to use a similar syntax to what you saw with the += operator. Now going on back, let's add another event. Let's say that we perform some work, and then when we're totally done with the work, we fire another event called WorkCompleted or something like that. Well, we can do the same thing, and in this case, it's going to look like the following. Notice EventHandler. Now, where did that come from? Well, let's go ahead and right-click and Go To Definition, and you're going to notice it's a built-in delegate, and now we're starting to get to our object sender EventArgs e, but this is a built-in delegate, this is the one that when you work with a button, for example. Well, that's using the EventHandler delegate. The EventArgs is empty, in that case, there's just a button click, not much else is being passed as far as data, it's not like a mouse event, for example, but that's one that's built in to .NET itself that we could also tap into, and of course, now if I do worker., we could do either WorkPerformed, and notice the lightning bolt here on each, or WorkCompleted, and then we're going to have to wire that up to our EventHandler, and that way when the delegate is invoked through the event, it'll know to call the function. We'll do that a little bit later. So now we've defined some events, we probably want a way though to maybe raise them as work is being done, so let's do something like this. Let's do a public void DoWork, and I'm just going to go ahead and tab here, and notice that there's this OnWorkPerformed. I'm going to go ahead and take that out for now, and notice there's an OnWorkCompleted because that's what we're going to cover in the next section.

Raising Events

Once you've defined an event in a class, you need a way to raise the event, and we're going to look at that here. Now, there's a few techniques that go way back, one of which I'm going to show you shortly, but there's a more modern technique that we should use these days, and I'll show you that as well. So getting back to our visual, we've defined our delegate, we have an event, but we don't have a way to send the event data up to the event handler, so let's take a look at how we can do that here. So the first technique you could use looks like the following. You'll notice at the top we have our event, but then we have this if conditional statement. If our WorkPerformed event does not equal null, then it's okay, somebody is attached and wants to know about the event, so we can then raise it much like you would do with the delegate scenario that I showed you earlier. Notice, we're simply taking the name of the event and then we're passing the args that match up with the delegate, in this case, an int and that work type enumeration. Now, this is an older technique, it still works, it's been around for a long time, and you're probably going to see this if you go search on different areas of the internet, but there is a more modern technique, and it's a little bit less code as well that you can use. It looks like the following. We can simply take the name of the event, and we're going to use the nullable operator here, the question mark, and if it's not null, we're going to call Invoke, that invokes the event, and then we pass the data, the int, and the WorkType enum value. That's going to go through the delegate pipeline, which is our WorkPerformedHandler, and whatever function is hooked up, then it would get this data. So if we put this all together, we have the delegate at the top, that defines what type of pipeline we have, we have a pipeline that can handle an int and a WorkType. We have our Worker class, it has that WorkPerformed event, and then notice we have this DoWork. Now the DoWork, this is just your normal method where something's happening, we're just going to do a simple loop, and as it goes through that loop, we're going to raise the WorkPerformed event so that you could know I'm on step one, step two, step three, whatever it may be, the listener could get that data. Now, notice what we're doing here though. We're actually putting a little wrapper function, it's called OnWorkPerformed, we're passing the int and the workType to it, and then you'll notice that OnWorkPerformed is marked as protected virtual. Why? We're making it overridable, that way if somebody is using the Worker class and they don't like how the event is being raised or they want to tweak the data before the event is raised, then they could go ahead and override this if you wanted. Now, we're not going to override in our demo, but I do want you to notice the WorkPerformed, we have the null operator, and then .Invoke. We pass the int, we pass the WorkType. This is a great pattern that you can follow for all

events in classes that you might write. We want to raise the event somewhere and that might be in multiple places throughout the class, by the way. Well, now we have a single method, `OnWorkPerformed`, and it has the sole responsibility of actually invoking or raising that event, so let's jump into the code on this. Earlier, we created the worker class, and we added two events and then we just have a normal method that's doing some type of work, but as it's looping, we'd like to notify a subscriber to this event `WorkPerformed`. Now because this will run super, super fast, I'm going to go ahead and do something that we don't want to normally do. We're going to add a delay on purpose. Don't try this at home kids, but it's good for demos because that way we can see the event being fired. And then I'm going to add the `OnWorkPerformed`, and we're going to pass in whatever `i` is, we're going to add a `1` since we're starting at `0` here with `i`, and we're going to pass the `WorkType`. Now coming on down, I'm going to go ahead and just paste the exact same function that you saw earlier. We have our protected virtual, it's overridable, if you will, and then notice we have our `OnWorkPerformed`, and we're going to accept the exact type of data that our delegate provided. Now, normally, in this case, I'd probably move the delegate since it's being used with the worker into the actual worker file, although it doesn't really matter where you put it, but we're going to do that in this case. And then once this is raised, notice it's going to call `OnWorkPerformed`, and then we're going to invoke it, we're going to pass the data. That is how we can raise the event. Now, could I take this line of code here and just put it right up top in a loop? I certainly could, but this is a pattern where we're putting the event raising code in a single location, so it's reusable throughout the class if need be, and that's the normal pattern we're going to follow that you see here with the `OnWorkPerformed`. Now, we also have this work completed, so let's add that as well. I'll say `OnWorkCompleted`. Now, this one doesn't take anything. It was the event handler if you remember if you look up top here. So I'm going to do pretty much the same thing. We're going to grab this, paste it down. We'll change the name of it to completed. This one isn't going to take any data. This is just notifying whoever subscribes that, hey, we're done. The work has been done and they can know about that. So now we can come on in, we can grab our `WorkCompleted`, and then we can invoke it, but what we're going to do here is we're going to pass the sender. Okay, and that's going to be this, that represents this class, the worker, and then we don't have any event data, so we're going to pass `EventArgs.Empty`, that way whoever subscribes to this `OnWorkCompleted`, they'll know it was completed, but they're not going to get any additional data. Now, if you wanted to change that, you could certainly make your own delegate. Recall that we're

using the built-in .NET one, event handler here. It just has the EventArgs, but I'll show you a little bit later how we could even make a custom EventArgs, and we're going to look at that in the next section actually. Before moving on, you might have noticed, though, that in the OnWorkPerformed, we're kind of hardcoding the value 8. We need to change that, of course, because when we call OnWorkPerformed, we're passing in $i + 1$, in this case, that represents our hours, so we need to change that to hours to fix that little bit of code. So now that we've done all this, it's time to learn more about the EventArgs.

Creating an EventArgs Class

The delegate pipeline you've seen so far allows an int and a WorkType to be passed, but that's not really the standard signature when it comes to events in .NET. In fact, you saw for the WorkCompleted event, we used event handler and it took a sender and an EventArgs, and that's what we want to do for our custom delegate, so we're going to look at that in this section. So if we go back to our visual, we've talked through the delegate and all the features of that, but when it comes to the event raiser, the normal type of event data you're going to see is that sender and then either the standard EventArgs or a custom EventArgs. In order to make that happen, we need to change the delegate's signature for the parameters to match the standard that .NET normally uses, and again, that's a sender and an EventArgs. That way, the event handler always gets the same two things that sender and then either the normal EventArgs or a custom EventArgs, and it's very predictable. As an example, earlier in the course, we looked at a standard button click event, and we saw that you get an object sender EventArgs e. Now, in this case, it's using the event handler delegate, which means the EventArgs are just an empty EventArgs, but you're going to see we can change that. We can make our own custom EventArgs class so that when we have custom delegates, custom events, the subscriber to that the event handler of that event can always count on getting these two parameters, so here would be an example of doing that. We have a WorkPerformedEventArgs class and notice it has an hours and a workType. Now the hours is the int, the workType is the WorkType enumeration that we saw earlier. So all we're really doing is taking those two parameters that we had before and we're just wrapping them in this custom WorkPerformedEventsArgs class. Now, I also added a constructor here, WorkPerformedEventArgs, that you could pass in those two items and then it'll automatically map those up to the properties that you see up above, but that's certainly optional. You

don't have to do that if you don't want to. The whole point of this, though, is that an event may fire a lot of data. What if an event had 10 parameters it passed? Well, that would be a lot of parameters to put in the delegate signature and the event handler signature and all those things, so instead of doing that, we're always going to pass a sender and some type of an EventArgs. Now, that EventArgs could have these 2 values that you see here or it could have 10 or 20 or 50 if you needed to, but it's always consistent no matter how the events are fired throughout the .NET app. Once we have the EventArgs in place, we can then change our delegate. We could say object sender and then WorkPerformedEventArgs, and those two pieces of data, the int and the work type, would simply be inside of that object that's passed. Now, there's even a more simple way to do this, though. In fact, all this delegate stuff we talked about, it turns out you don't actually have to write your own delegate. I covered that with you because it's important to understand what a delegate is and how you would define it, but .NET does have something built in that you could use out of the box, and it's a generic. So rather than defining our own delegate as we've done up to this point, we could do this. When we define the event, we can use an EventHandler of T generic. Now, EventHandler is the EventHandler that has the center in the EventArgs, but notice in the generic, we're passing work performed EventArgs. Now, it'll substitute that in when it generates the code and it's like writing our own custom delegate, the one you saw above, actually, that's crossed out right now. So normally, you can get away with just using the EventHandler of T, creating the custom WorkPerformedEventArgs or whatever your EventArgs class is named, passing that to the generic here, and then you're done. You don't actually have to write the delegate. So now that we've talked that through, let's go back to our code and see if we can first off create a custom EventArgs class, and then we're going to modify our event to use EventHandler of T. Coming back into our Worker class that we created earlier, we have our delegate, and we could come in and change this to object sender and then our WorkPerformedEventArgs, which I'll show you in a moment, and we could call it e for the EventArgs, but we're going to go ahead and use the EventArgs of T, the generic that I just talked about, so let me just comment this out, that's going to break this for a moment, but we'll come back to that. Now, I've added a new class called WorkPerformedEventArgs you'll see in the Solution Explorer, and it's exactly what I just talked you through. We have the int hours, the work type, and then I have a default constructor here so that we could easily pass this data in and initialize our properties with the values. So to make this work, we can go on back and we can change this to EventHandler of and then our class, so let's do that. We'll

do EventHandler and then you'll notice the second item here is EventHandler of and then this is a generic RT, and we're going to call this WorkPerformedEventArgs, and then we'll add our operator there as well. Now things change a little bit because now when we call OnWorkPerformed, we are going to pass the int hours and the WorkType enum, but we need to change this because the invoke of WorkPerformed, it no longer takes those two parameters, it takes object sender and then our custom WorkPerformedEventArgs. In fact, if we right-click and go to definition of our generic, notice that we have our T, TEventArgs, there's our object sender, and then whatever we put in the generic parameter is going to get substituted in here as it does the build, and for us, that's going to be the WorkPerformedEventArgs, so they've kind of done all the work for us and they're just using generics to make that easier for us. So we can come on in, and instead of this, we can do the following. We can say Invoke and we know that we're going to pass a sender, and then we can use the constructor of our WorkPerformedEventArgs, so I'm just going to paste that in, and we could say a new WorkPerformedEventArgs, give it the hours from here and the workType, and we're good to go. So you can see it cuts back on the code a little bit, which is nice, but if for some reason you wanted to use your own delegate, you certainly could. We could come in and add the object sender and then the WorkPerformedEventArgs here if we wanted and that would be fine, but now we're using the built in .NET features, and it makes it a little bit easier to maintain, I think, down the road and we're kind of good to go. Now that wraps up adding delegates, events, raising the events, custom EventArgs, all those pieces are now in this little bit of code here, but if we go back to our program, notice we create the worker, but we never ever did anything with the events. Well, we have a little bit more to talk about there, so we're going to cover that in the next module. So before we jump into that, let's review this module and then we'll look at the event handler side of the equation.

Summary

To wrap up, we've taken a look at how events are associated with delegates. They both go hand-in-hand. In fact, without the delegate pipeline, events wouldn't be that useful actually. We know that the standard event signature in .NET accepts a sender and an EventArgs. Now, if you just go with the event handler delegate, that EventArgs is empty, but we looked at how we can write custom EventArgs as well. Earlier in this module, we also looked at how the parameter signature for an event handler has to match the delegate signature. We're going to look at that even more in the upcoming

module. And finally, we saw that while you can certainly write your own custom delegates, and there's nothing wrong with that at all, .NET has a built-in EventHandler of T generic that provides a really simple way to create a custom delegate, all you have to do is substitute in your custom EventArgs class, that makes it nice and easy. Now, the remaining part of the puzzle that we touched on just a little bit in the module, but we need to dive in deeper from an event perspective anyway, is handling the events, so we're going to look at that in the next module.

Handling Events

Introduction

This module is all about handling events. In the previous module, we did look at instantiating a delegate and then pointing that delegate to a method. That wasn't really an event handler, though, it was just a function pointer. So now we're going to expand upon what we learned earlier, build upon that knowledge, and frame it with regard to events. So we're going to start things off by talking about how we can instantiate delegates and then point them to event handlers. Now it's going to look almost identical to what you saw when I first introduced delegates at the beginning of the last module, so the good news again is we're going to build upon that, but it's going to be a little bit different in that the event is going to fire which triggers the sender and the EventArgs to flow along the delegate pipeline to the event handler, but under the covers, the delegate is still acting as a function pointer, so everything we've covered up to this point, it still applies. Now, once we get through that, we're going to talk about something called delegate inference, and this is a nice compiler feature that makes it so you don't have to write as much code. In fact, this will tie nicely into what we'll be covering in the last module of the course on lambdas, but I'll introduce a little bit of that here and then we'll dive in deeper as we move along. Now, delegate inference is important because back in the old days of .NET, you actually had to physically write the code to instantiate the delegate, hook it up or attach it to the event, and then give it that function that you want to point to, and it could be a little bit of extra code that felt redundant in many cases. Now, we don't have to do that. For quite a while now, the compiler has this built-in delegate inference feature, and in a nutshell, it just means you write less code, and that's a

good thing for maintenance and your productivity. So with that, let's go ahead and dive into instantiating delegates and handling events.

Instantiating Delegates and Handling Events

We know that delegates play a key role with events in .NET and that we have to instantiate a delegate in order to give it the function or the method that we want it to point to. We're going to look at that here. Now, I mentioned in the intro to this module that we've already covered some of this, so hopefully this will be a little bit of a review for you, which is always good for learning because you'll see something again. Now we've seen this visual many times up to this point, so I hope you're pretty comfortable with it now. We have the event raiser, we have our delegate and our event handler, now we're going to focus on the event handler side of the equation, and that means that as those EventArgs, as well as the sender goes up to the event handler, we can get to that data and use it. So these are the two areas we're going to focus on in this section. Now, whether you're using a custom delegate or you're using something like event handler of T, either way, you're going to have a sender and some type of an EventArgs. In our example, we've been using the WorkPerformedEventArgs, but this is something you could completely customize depending upon your scenario. You might have, for instance, a customer ordered event and maybe the custom ordered EventArgs have details about the order as properties within that EventArgs class. Lots and lots of scenarios where events could come into play, but the important thing here is that we know that whether it's a custom delegate or an EventHandler of T delegate, the signature between the delegate and the event handler has to match perfectly. So notice in this example, we have this object sender, that's for our delegate, and then in this Manager_WorkPerformed, we have an object sender. We have a WorkPerformedEventArgs, and we have a WorkPerformedEventArgs those have to match up, or of course, the compiler is going to throw an error, and you'll sit there scratching your head as to why it won't build. Now, how do we actually hook an event to a callback function or event handler? We use the delegate. So this is what we saw earlier, although I just used the delegate directly in the previous module. Now, notice we're saying worker. and there's our event, WorkPerformed and then we're going to attach a pipeline, which is our EventHandler of WorkPerformedEventArgs, so this is not using the custom delegate, we're using the built-in generic and .NET, and we're going to point that to a worker.WorkPerformed event handler. Now the important part here in addition to the delegate and the event and all of that is the +=, this operator

is going to add to an invocation list. If we attach the delegate, which is the EventHandler of T to the work performed, then anytime that event fires, that delegate pipeline will be invoked and the event handler would be called, of course. Now, there's also a -=. What if I want to stop listening? Well, then you could say -= and give it the delegate and the EventHandler function, and those must match by the way, that would remove it from the invocation list. So under the covers, think of it as like a little notebook that every time you do +=, it adds the EventHandler function Worker_WorkPerformed, in this case, to the list. If I do -= and give it the same setup, it'll remove it from the list, and that's how events and delegates work under the covers. So in this example, the Worker_WorkPerformed might look like the following. Notice again, we have the signature of object sender WorkPerformedEventArgs e and then it's up to us to do something once that event fires in that particular method. So let's jump over to the code and do something like this to actually listen to the event. So earlier in the course, we created our worker, but we never subscribed to the event, so we can do that by saying worker. and then notice we have our two events here, WorkCompleted and WorkPerformed. Let's go ahead and do WorkPerformed here, and then it's already given me something that we're not going to jump to quite yet, so I'm going to do this much like you saw earlier, and we're going to do a new EventHandler of WorkPerformedEventArgs, and then we need to give it a callback or EventHandler function, whatever you'd like to call it there. So let's say that we had something like you saw before. We have maybe a static void Worker_WorkPerformed, something like this, and we're going to give it that method name right here. Then we'll go ahead and finish that off. Now, you'll notice there's a little green underline here, and it's going to tell us that there's a nullability option here, and the reason is that it's possible this could be null, so we'll fix that by adding the nullable operator onto the object there. Alright, so when this runs, what should happen is every time this event fires WorkPerformed, it should call and write out the hours and the work type. Now, if we go back to worker, recall that we're going to pass in a number of hours here for DoWork, so we're going to need to call that, and we're going to pass in the WorkType, but every time this loop fires, the event fires, and so if we go on back here, once we've hooked up the event, we can now say worker.DoWork, and then we could give it the number of hours, let's go ahead and say we're going to work 8 hours on generated reports. Sounds very exciting. So let's go ahead and run it, and there we go. Now, there's that delay to kind of simulate the loop, but we should get up to 8 here and then we should stop, and now we're done. Now, how do we know when we're actually done? Well, recall that in addition to WorkPerformed, we also have a

worker.WorkCompleted. Now, I'm going to do the same thing here. I'm going to say new EventHandler. Now this is the non-generic EventHandler, and I'm going to give it Worker_WorkCompleted, and then we'll go ahead and make that function down here as well. Let's just copy this one, and we'll go ahead and grab this code and update it, but this is not going to send a WorkPerformedEventArgs. This is going to just send the EventArgs, nothing in there, it's empty, and so we're just going to write out Work has completed. Alright, so now when WorkPerformed is called, it'll call our worker_WorkPerformed. When WorkCompleted is called, it'll call our Worker_WorkCompleted. Let's go ahead and save that and run it. There we go. So once we get to the end of the loop, which is at 8, we should see another message here, and there we go. Work has completed. So what's so great about this is now even some very specific tasks that are going on in your code, if you, for whatever reason, need to know about each part of that task, you could write your own custom event and notify another part of the app, maybe you need the log, for example, each little thing that happens. Well, an event could fire, that could cause some logging event handler, and then you could write to a log or a database or something like that. This is one way to do it, but there is another shortcut way we could do this and that's through delegate inference. Let's look at that next.

Delegate Inference

When you attach a delegate to an event, there's a shortcut syntax you can use and this is enabled through some compiler tricks. Now, delegate inference has been around pretty much since dinosaurs roamed the earth. It goes all the way back to the early, early days of C# and the older .NET Framework, so what's nice about this is you don't really have to do as much code, you can simplify the amount of code you write, and it's very, very easy to take advantage of. Earlier, I showed hooking up worker.WorkPerformed through the += operator to an EventHandler of T delegate, and then we gave it the EventHandler function or callback function. Notice, in this case, I hooked the += directly to the Worker_WorkPerformed method, which you can see below, and it'll just invoke it as the event fires. Now, there is a delegate under the covers, it's just that the compiler is taking care of that for us. We don't have to worry about it, and that's really all delegate inference is. Because the event knows the type of delegate because it's the type of the event is the delegate, it can automatically infer that the method on the right of the += must match the signature that the delegate defines, in our case, an EventHandler of WorkPerformedEventArgs, so we know we're going to get that sender and that

WorkPerformedEventArgs, of course. So this is just a really nice convenience feature where you write less code and it means you don't have to worry about as much, so let's look at changing that in the code. So rather than having the new EventHandler of WorkPerformedEventArgs or EventHandler of T, we could just hook this directly, and likewise, instead of the work completed, hooking to the new event handler, we could just hook it directly to the function, really, really nice, simplifies a lot of the code, don't have to worry about as much, and again, the reason this works is if we mouse over the event, it knows that it's an EventHandler of WorkPerformedEventArgs. Therefore, the method that it's going to call, the compiler can just automatically generate the necessary code to make sure that the delegate pipeline calls that particular method, likewise WorkCompleted is an event handler, that's the built-in .NET one, and it can know that okay, well, that one only takes a regular sender and a good old empty EventArgs as you see down here, so it cleans up the code quite a bit. Now, the last thing I want to show you on that is you might have noticed that when I did the following earlier that the editor tried to help me out a little bit, so if we do worker.WorkPerformed, and then +=, now it's kind of stuck on this, but watch what happens if I start to put a parenthesis here. This gets us into lambdas. Now, I'm not going to go into it now because we haven't covered it yet, but that's going to be the next module of the course. We're going to learn more about this syntax, some generics that are built in we can take advantage of, and some other really cool features in the C# language. Before we jump into lambdas though, let's do a really quick review of this module, and then we'll move on.

Summary

Throughout this module, we've talked about event handlers. They're very simple to use. All you have to know is that under the covers when an event fires, the delegate pipeline gets invoked, that points to the function, the method, the event handler, whatever you'd like to call it, and then we're off and running, we can then act upon the data we get. So we learned that the signature of a delegate and an event handler must match exactly, otherwise, you'll get a build error. And then to listen to an event, you first have to construct a delegate and add it to the invocation list, and we do that with the += operator. But you saw that rather than having to instantiate the delegate and assign it and all of that, we can use the C# compiler feature called delegate inference to have it automatically generate the delegate based upon what event we're hooking to the event handler. It does a great job at that, it knows what to do, and it just works. So now that you've seen event handlers and we've put all the big picture together of

events and delegates, EventArgs, event handlers, all the things in between, now it's time to switch our focus to a new topic, which is lambdas. We're going to first look at what lambdas are and then we're going to look at some generics that can help, such as Action of T and Func of T, TResult, so I'll see you in the next module.

Lambdas, Action<T>, and Func<T, TResult>

Introduction

C# has a lot of great features throughout the language, but one of my all time favorite features is lambdas and that's what we're going to be talking about throughout this module. Now, lambdas have been around for a long, long time. They're one of the early features that made it into the language, and I still remember when I had the opportunity to go to Microsoft and hear directly from Anders Hejlsberg, the creator of C#, about lambdas and some other related technologies, and I was just blown away at the time. And even today, I still think it's a phenomenal feature. I use it all the time. So we're going to talk about what they are, and then we're going to introduce some helpers, if you will, Action of T, Func of T, TResult. Now these are generics, but they can also help out in situations where you might use lambdas, and we'll talk about that a little bit more. So we're going to start things off by talking about what our lambdas and how can they relate and be used with delegates. We'll revisit what we've built up to this point, but we'll convert some things to lambdas. From there, we're going to talk about what Action of T can be used for, very, very useful in certain circumstances, and I'll explain that, and likewise, there's a Func of T, TResult, and we'll talk about what that is. Now, if you're new to this, I still remember the first time I saw Func of T, TResult. At the time, I was a little bit newer to generics and some of those features and it looks kind of intimidating at first, but it's not, it's actually very straightforward to use. I'll have some nice examples to show you why you would use it. And by the end of this module, you'll have a really good idea about what lambdas are, why you would use them, and then these generics as well. So let's jump right in.

Understanding Lambdas and Delegates

Lambdas are a lot like shortcut methods. In other words, instead of writing a standalone method like we would normally do in a class or other area of C# code, we instead, can actually write that and attach it directly with the delegate, with the event, or even in other situations. Now, the first thing we're going to do is talk about what is a lambda, what is the syntax look like, and then I'll show you a few examples, and then we'll expand upon that by showing other scenarios where some of the generics like Action of T and others can be used. So here's an example of a `SubmitButton.Click`. Now earlier in the course, we saw that when we want to attach the event to a delegate and then have that point to our method, our event handler, we normally write a separate method, but you don't have to, you can do the following. Now, this is using lambda syntax and a lambda operator, so let's break this down. First off, normally, when you call a separate method, that method has a name and then it has parameters. Well, in this case with lambdas, we don't give the method a name, but we do define the parameters. Now notice in this case, `s` would represent our object sender, `e` would represent our `EventArgs` `e`. Now what's happening here is the types for those parameters are being inferred by the compiler, so we don't have to add those types, it's a very shortcut syntax, and that's the actual parameter signature for this inline method, if you will. Now the next piece is the lambda operator, and really this is just a separator. I almost think of it as like a bow and arrow scenario where the parameters are like apples on an arrow that you're putting or fire or whatever you want to put on your arrow, I guess, and they're going to shoot those off to the area on the right, which is the method body. Now, in this case, because the sender is the button and the `e` is the empty `EventArgs`, there's not much to do, except act upon the Button Clicked, so in this case, we're just going to do a `MessageBox.Show`, but you're going to see that with other scenarios where the parameters have some data, we can, of course, get to those parameters and actually use them, so that's a general overview of what a lambda operator looks like, and again, just think of that little arrow as it's shooting those parameters over to the body on the right and it's just a separator. Once I realized that I went, oh okay, it's not like some math operator or something like that. It's just a separator between the parameter signature, the `s` and the `e`, and the body over on the right. So let's look at some other examples. So here's an example of a delegate. Now, we've looked at delegates all throughout the course, so hopefully, this looks fairly familiar, but notice we have a delegate that returns an integer, and then the signature that the delegate might invoke takes two integers, `a` and `b`, you'll see, so very

simple. You pass in two things, we can add those together maybe and it returns the result. Now, if you look in the main, you'll notice that we have our ad delegate, we called it ad, and then we're using a lambda. So what we're saying is that the callback function, the function pointer, whatever you want to think of it as, has a signature of it takes two parameters a and b. Now, we could name those differently, by the way, it doesn't have to be a and b, they don't have to match the names of the delegate signature, so if you wanted x and y, for example, that would be fine, but we're going to do a and b. We have the separator, so the parameters get shot over to the right with the bow and arrow if you like that analogy, and then we're going to add those together. Now you'll notice there's no return statement because we know that the delegate returns an integer. Well, that's implied with this type of a lambda. When it doesn't have any curly braces, then if there's a return type for the delegate, in this case, then you don't have to type the return keyword, it's just implied. So now if you look at the next line, you'll notice we're going to add 1 and 1, and of course, we know that equals 2, so what'll happen is 1 and 1 get passed in for the a and the b above, the little lambda operator shoots that over to the right, it adds those two together, and then automatically returns the result. Really nice, really compact, and if you're wondering, do I have to use this? Could I just make a standalone method? Yeah, of course, you could. We've seen that throughout the course up to this point, actually, but this is a syntax I generally prefer in many scenarios, and I'll show you more about that. Now, here's another example. Now this delegate doesn't take any parameters. We have a log delegate, but it does return a Boolean probably to say if the log was successful or something like that. In the main, we then create our log delegate, ld. Notice it has an empty parameters. Now, if you think of the bow and arrow scenario, we're not putting anything on the arrow before we shoot it, it's just empty. It's just an arrow. We're going to shoot it over to the right, we in this case, write to an EventLog and we return true, but notice it's a shortcut method. Now, when we call ld and get back the status in that last line of code, we don't pass any parameters because there are none, but it writes to the EventLog and then it returns true. Now, obviously, you might return true, false, or an error. This is very simplified code. But those are all examples of using lambdas, so let's go ahead and jump back into the code we've been looking at, and let's see how we can actually use lambdas with our delegates.

Lambdas in Action

Let's take a look at lambdas in action. In this example, I've opened the project in VS Code, but if you're using Visual Studio, feel free to open the csproj or the solution file, but you'll notice that we have the same exact code from the previous module. We're right where we left off, we have our Worker, our WorkPerformed, and our WorkCompleted events, and those are each calling event handlers and we're using delegate inference. We don't have to have standalone functions though. We could actually do this ourselves with a lambda and actually shorten up the code if you'd like. Now, this is an option. Sometimes I like to do this, sometimes I don't, it really depends on the scenario and the size of the method. To show the lambdas in action, let's convert this one to use a lambda. So instead of calling the standalone, what I'm going to do is pass the little shortcut for the parameters. We're going to do s, e, we're going to put the lambda operator, and I'm going to write out two lines to the console, so I'm going to put a curly brace because of that, and then I'm going to just paste in some code here for what we want to write out to the console, and we're going to write out something like the following, and so you'll notice we're going to write out Worked, number of Hours, hours doing, and then WorkType, and then some other value just to show that we're going to have multiple statements in this particular lambda. So to review, when the WorkPerformed event fires, we attach to the delegate. Now this is going to use delegate inference again, and we have the inline lambda method. It has two parameters. WorkPerformed, we know, had a sender and a WorkPerformedEventArgs, so this is an object and this e is our WorkPerformedEventArgs, it can infer that. We have the lambda, which is the separator, and then we have the body. Now in the body, we can do whatever we want. In this example, we're not returning anything, but because we're using the curly braces here, the brackets, if we did want to return something for whatever reason in another scenario, we would have to put the return statement. The only time you don't have to worry about that is when you don't have the curly braces and it's a single line lambda. Then if there is a return type, it can automatically infer that return for you like I showed with the add delegate earlier. Now let's also convert this one, and this just has the object sender and the regular EventArgs built into .NET, so s, e again, this is more of the empty EventArgs, and I can do a single line here. I'm just going to say Console.WriteLine, and we'll write out Work is complete. So now we've converted our events for WorkPerformed and WorkCompleted to, instead of calling standalone functions down here, to actually call inline lambda functions. Now again, either of these approaches is going to work for you. It really depends on what you want to do. In some cases, I like to use the inline, in some cases, if the method is really big, then I generally prefer a

standalone method. Now, in addition to using lambdas like you just saw with the delegates in the events, we can also use lambdas to query objects. So I have a customer that's added into this project and it's a record that has city, FirstName, LastName, and Id you'll notice. So I'm going to paste some code in here to create a really simple List of T, let's format that, and you'll notice it's just creating John Doe, Jane Doe, Suki, and Michelle Smith. Alright now, what I'd like to do is use a lambda to actually query this, so I'm going to do pretty much what you see right here. I'm going to say `phxCusts =`, and we're going to say `custs`, our list of customer dot, and now I'm going to break this into a separate line and hit Tab, and notice that we have these extension methods. Now, these aren't new, they're not unique to lambdas per se, these are extension methods for List of T, been around for forever in .NET. Now the `Where` is going to allow us to add a predicate, a `Where` clause to say, hey, I only want to find Customers, so the `C` here represents each type, which is a customer record, where the city is Phoenix and the Id is less than 500, so you'll see that we should get two in Phoenix, but only 1 is less than 500, so John Doe should be what's displayed once we write this out. Then I'm going to `OrderBy` just in case I had more than one, in this case, looks like I only have one. Now, this is using lambdas though, so what'll happen is it's going to evaluate each of these customers, so the first one passed into the `c` here would be the John Doe record. We check the `c.City` is Phoenix and the Id is less than 500. Now, if that matches, then it's going to include it in our `phxCusts`. Then the next customer gets passed in, Jane Doe. Does the city equal Phoenix? Yes, but the Id is not less than 500, it is 500, and then it just keeps doing that, and that's what the lambda is doing, so it's providing this really nice inline way to do an inline evaluation function, a `Where` clause, if you will. Now, when it's done, it's going to say grab each of the customers in `phxCusts` and let's sort those by first name. Now we're only going to get one, I believe, but we'll run this in a moment. Now, once we're done with that, we can go ahead and we'll just do a standard `foreach` loop here, and we'll save our `cust` in `phxCusts`, and let's write out that `FirstName`, so that looks pretty good, and let's go ahead and try this out. Alright, so let's do `dotnet run`. Now, you'll see that the first thing it did was find our customer, which was John. We go back up, John was in Phoenix, and the ID was less than 500, so it looks like that lambda with the `Where` and the `OrderBy` worked, and then we see the other code that we had down here, which you've already seen in previous modules for our `Worker` and the `WorkPerformed` and the `WorkCompleted` event. So that's an example of how we can actually use lambdas in multiple scenarios. One is obviously just to replace standalone methods with inline lambda methods. The other is to actually query objects like lists or

arrays or others so that we could actually use lambda syntax to do Where clauses or OrderBys or others, very, very powerful if you haven't played with that. So that's a look at getting started with lambdas, but there's a little more to the story as well, so let's look at some other things we can do too.

Using Action<T>

[Autogenerated] Up to this point, you've learned how to write your own custom delegates and even seen the event handler of T for writing delegates and hooking them up to events. But there's some other delegates built into the .NET Framework that you can use out of the box. And one of those is action of T the other is called funk. And we'll look at that in the next section. So we know delegates are nothing more than pipelines or function pointers. And .NET provides several of these out of the box. So the first one we're going to look at is called action of T. Now it accepts one or more parameters, but it doesn't allow for a return value. So you would use this in situations where you want to pass some data in perform some action, but you're not gonna return anything. Now, the other one we're gonna look at in the next section is funk of TT result. Now this is very similar to action but unlike action where you can pass in multiple parameters, but it doesn't return a value funk of TT result, the T result kind of gives this away accepts one or more parameters but it can also return a value that value will be defined by you. And it's the t result that you'll pass into the generic. I'll show that a little bit later. Let's talk about action of tea though and show an example of it and then we'll jump into a demo. So in the main, you'll see that I have an action of string. So what this is doing under the covers is creating a delegate that only accepts one parameter. It's a string, no return value. We're gonna call this message target. Now, if the arcs that are passed into main are greater than one, I'm gonna say that message target is this show Windows message. And you'll see that method down below. Else if the arcs aren't greater than one, the message target is gonna be console dot right line. And so when we invoke the delegate, whatever string value you pass in, it's either gonna go pop up a message box or write to the console. So the last line of code, the message target invoking action that you see. This is the same as all the other code we've seen up to this point where we invoke a delegate or an event that uses a delegate. This will invoke the function which would either be show Windows message or the right line and then pass the string, which is that single parameter again, invoking action. So instead of us writing the delegate adding the parameter, that's a string, we can just use action of string. And I'm gonna show you in just a moment that you can also pass multiple parameters into action, but it never returns

a value. It's always a void. Now that you've seen a simple example of action, let's look at another one in code. So coming back to the project for this module, you're going to notice that I have a process data class. Now at the top of that class, I have a custom delegate called BIZ rules delegate that takes two parameters and returns an in. So you can see it takes two integers X and Y and then it's going to return an in. Now, we could use this directly. We're going to initially or we could use some of the built in delegates. Now looking down in the process data class notice first off that I have a process function, it accepts an X and A Y as well as a delegate. Now, why would you pass a delegate to a method? This gives a lot of flexibility. Let's say that there's five different methods that your app might call in particular situations. But you never know the names of these. They're kind of dynamic. Well, you might initially think, well, I could do an if statement or a switch statement or something like that, but then you would be limited to knowing exactly the ones that are gonna be passed in. Well, this allows us to have a placeholder and basically say, pass me any method that accepts two integers and returns an integer and then notice we're gonna invoke it, get the result and then write that out. Now, that's one way to do this. Now, in cases where you want to do an X and A Y, an in and an in but not return of value. Here's another option. Look at this, here's our action of T. Now recall that action of T accepts one or more parameters of the type T. So if you said action of ant comma string, then it would be two parameters. What the first one is an ant, the second one is a string. In this case, we're gonna say you can pass me two integers as the parameters. Now again, this is dynamic. We don't know what's going to be passed in at this level. All we know how to do is invoke whatever method is passed in for the action pass in the X and the Y and then it might log those, but it's not going to return a value. In this case, all we're gonna do is write out action has been processed. Maybe it updates a database as an example. There's another one that we're going to look at the next section. You'll see that below. That's funk of TT result, but we'll get to that momentarily. So let's see how we could use these. Let's go back to our program and right under where we did the lambda with our customers list. I'm gonna paste in some code here. So notice we're gonna new up our process data. Now, the next thing I'm gonna do is create a BIZ rules delegate that allows you to pass in an X and A Y and then this has the implied return statement here. So we're gonna use our custom delegate right now and then this one's gonna add. But notice the second one multiplies. Now in our data dot process we're gonna pass in multiply deeg, OK? Now, that would be one way to do this. That's just using the custom delegate. We're not really using the action at all because the process function here is still using our custom delegate, but we can

add some other code to show off how we could use action of T, all right. So notice in this case, we have an action of intent. So a delegate with two integers but no return type. Now, in this case, the X and Y are passed in to our little lambda here and we're gonna ride out X plus Y. But on the multiply, we're gonna ride out X times Y. Now, this is a little bit different because our delegate, if we look back at it again, it did return an integer. You can see that right there. Whereas in our action, we invoke it, but we don't return anything. That's why we're just gonna write out the X plus Y or the X times Y. Now we're gonna call process action. So if we pass in two and three, it should write out two times three, which is of course six. So what's so nice about this is now process action is not limited to just the add in the multiply functions that you see here, you could have a whole bunch of functions that are dynamically passed in based on different scenarios, you would have to add a little logic of course, to do that. But now we are not locked in to just one type of function that can be invoked. In fact, if I asked you what is being passed in here, you don't know, we have no idea. We just know that it's gonna pass two parameters, but we don't even know what it's gonna do in this example. If we look back here, yeah, we know it's gonna write out to the console. But at this level, we have no idea if it's going to write to a database or a console or whatever, it's gonna do very, very dynamic and flexible. So now that we've seen that, let's go ahead and run it. So we'll run our .NET run and there we go. We noticed that our John has written out that's from our earlier one, but here's our six. Now that six is this one that's just using the custom delegate. And then the second six, this is writing out the console dot right line X times Y. Of course, because that's what we passed in to our process action and our process action took that custom delegate and then invoked it right there. Now, let's go ahead and stop this and going on back. Let's go to program CS and instead of passing my multiply action, let's go ahead and pass my action, which that one is gonna do the add. Now, let's run this again and see what happens here. Now, you can see that we got a five in this case, very sophisticated math here, obviously, that added the two and the three together. So that's now showing the add delegate. So whether you go with that one or the multiply, now you can pass one in dynamically based upon the needs of your program. So you can see that this allows for some really dynamic scenarios and we've kind of satisfied what we did up here with the custom delegate, but not quite right, because we know our custom delegate allows for a return type. Whereas action does not. Well, that's going to get us to our funk of it result. And we're going to look at that in the next section.

Using Func<T, TResult>

[Autogenerated] Another built in delegate and .NET is `Func<T, TResult>`. And as I've discussed up to this point, it's very similar to `Action` except it allows for the result to be passed back. So here's an example of using it. Notice we have a `Func<string, bool>` that's gonna be the input parameter. And `bool` that `Boolean` is gonna be the return type. Now, you can have multiple input parameters just like `Action<T>` but whatever the last parameter is in the generics list is gonna be your return type. We're gonna call this `LogFunc`. Now, if `ARG zero` passed domain is equal to `event log`, we're gonna assign the `log` to `event log` method to our `LogFunc`. And you'll see that down below. Else we're gonna assign the `log` to `file` method that you'll see down below. Now, let's assume that `log` to `event log` rises to maybe a system event log on a server. Whereas `log` to `file` rise to a specific file, the final line in `Main` is gonna call `LogFunc`. So that's either gonna call the `log` to `event log` or `log` to `file` based on what was assigned to our delegate, we're gonna pass in `log message` and then that's gonna return `true` or `false` our status. So you can see that this is very similar to `Action<T>` or even writing a custom delegate. But in this case, not only can we pass in one or more parameter values, we can also define the return value. So very useful in that case where you need that return value. Let's jump back into our code project and see this in action earlier, you saw how `Action` in in could be used. But that didn't really match up with the custom `Biz rules` delegate because `Action<T>` doesn't support a return type. Well, `Func<T, TResult>` does. So if you look down here at our `ProcessFunc` method, you'll notice that we have a `Func<int, int, int>` in in, in that may seem a little bit weird at first, but it matches up exactly with the `Biz rules` delegate. You see at the top of the file, the two ints are the `X` and the `Y` you could say and then the third in is the return type. So whatever the last one is in that list of generic types, that's always your `TResult`, your return type. Now what that means is we can now have a placeholder for any function that accepts two integers as parameters and returns an integer. We can then invoke that and actually get a result. Now, in this case, our `ProcessFunc` is void but we can certainly change that. We're just going to write it out in this case using `Console.WriteLine`. Now, if we go back to program right under the `Action`, I'll add some additional code and this will be for our `Func<T, TResult>`. So notice I'm gonna have a `Func<int, int, int>`, a delegate and that's gonna have an in, in, in again, the return type is the last one, the `int`. So we would pass in two values to add on the next one. We're gonna have the same exact type and we're gonna multiply the two that are passed in and then return that value. Now we're gonna call `ProcessFunc` pass in three and two. We're gonna pass in the multiply one which is gonna then put three and two together multiply

that and give us six. So if we run this, we should see the same exact thing is up above here. But in this case, we're actually able to get a return value and do something with that if we needed to. And there we go. So you'll notice we have a six, a six and then another one. So we have the first one is data dot process. That's our custom delegate. The next one is two times three with our action, but that didn't return the value, we just wrote out the value and then the final one is now passing that in multiplying the numbers and then we can get a value back as you saw right down here. So this is a good time to wrap up by talking about writing custom delegates versus using built in which one's better. There isn't a better, it's a little less code to use action of T or funk of tt result. But writing your own custom delegate isn't that much code either way. So it's kind of up to you on if you want to use the built in ones or if you want to write your own, but now you'll know about the options. And that's really, I think the challenge with programming in general is knowing what's available and knowing what tool to use in which scenario. So now you've learned about what we can do with custom delegates, action of t funk of t how we can use them with delegates alone or with events and delegates. There's a lot you can do here and knowing about this is certainly a really, really good skill to have in C#. So let's go ahead and do a final wrap up on this module.

Summary

[Autogenerated] Throughout this module, we've talked about the role of lambdas, what they are and how they can be used with delegates and events. They provide a really nice in line syntax that's concise. It's easy to use once you understand the syntax and it's something that I personally use a lot in my C# apps. So I'd encourage you if you're not using lambdas now to experiment with it, try to integrate those more into your code and see how you like it. They're certainly not required. But there are cases such as querying objects like we did with the list of customer that it is really, really useful. It's a nice tool to have in your tool belt if you will, once you understand how it all works. Now, in addition to lambdas, we also took a look at some of the built in delegate types that .NET provides such as action of T or funk of tt result. These are very useful if you don't want to write your own custom delegates. Now writing custom delegates is pretty straightforward once you understand how they work, of course. So this is kind of a trade off. Do you want to use the built in types. Do you want to write your own? I think everyone has their different views on that. Some people prefer the custom one because it's very clear. Whereas other folks might say no, I'd rather just use what .NET gives me and I'm really good at

generics. So i'll just use that out of the box. There's no right or wrong way on this. So it's up to you to choose what would you prefer in general. I do prefer to use things out of the box where possible, but I always like to approach every situation with an open mind and kind of evaluate what is the best approach for the team. Definitely something you'll have to do as well. So that's a wrap on this course. I hope at this point you have a really good idea about event razors, the role of delegates and how event handlers get data, how you can write custom events and even event arcs and get those passed through a delegate pipeline up to the event handler. Knowing these features of .NET is a really good skill to have. I can't say that I need this every single day because not every app needs a custom event or a custom delegate, but it's one of those skills that once you know about it, it just opens up your world of CRP even more. So I'd encourage you to try to integrate some of these features into your apps as appropriate of course, because they can be really powerful when used in the correct way. So with that, thank you so much for making time to go through this course with me, I've really enjoyed putting it together and trying to describe these topics in a way that's much more approachable and understandable. And I hope that was the case for you. In addition to this course, I have several others on plural side, ranging from Docker Container courses, Kubernetes courses, Angular and other front end courses and even some more. So definitely check those out if you're interested and I'll see you in the next course.